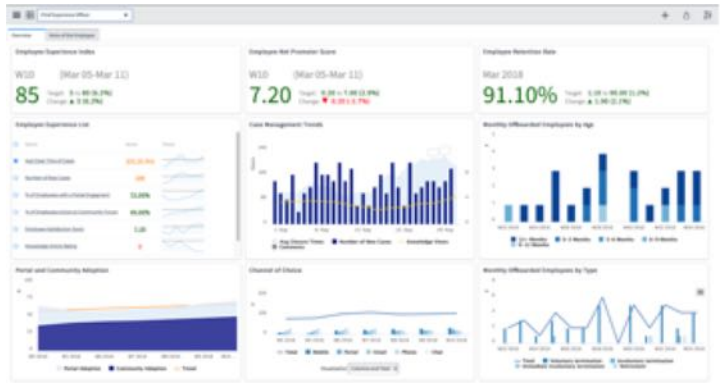
Analytics Dashboards



There are many tools on the market to help visualize data in graphs and dashboards.

An **interactive dashboard** is a combination of graphs, text fields, and controls allowing user to select the data, filter data (for example, by date range or by geography or by user), aggregate, compare. etc.

The data queries are usually formulated in SQL

Here is a demo of how fast Microsoft Power BI works with MS SQL Server - 500+ Mln rows of data
- https://www.youtube.com/watch?v=mQyFp2MSl-s

And here is a demo of Kinetica DB using Tableau & Power BI:
- https://www.youtube.com/watch?v=7XliLU7ZC_s

Notice:
- sub-second response on billions of rows of data
- not need to create indexes, no wait time, fast load (even live streaming) of data (modern databases use columnstore indexes - and are highly parallelized)
- https://docs.microsoft.com/en-us/sql/relational-databases/indexes/columnstore-indexes-overview?view=sql-server-2017

The top leaders for analytics platforms are **Microsoft** and **Tableau**.
Both use standard SQL to get data.
https://www.gartner.com/doc/reprints?id=1-65P04FG&ct=190125

Which database to use?

There are many options for SQL database.
For example, on Azure Cloud - Azure SQL Data Warehouse
- https://azure.microsoft.com/en-us/services/sql-data-warehouse/

BI & Power BI - https://powerbi.microsoft.com/en-us/

On Google Cloud we can use "Cloud SQL" (managed PostgreSQL) or BigQuery.

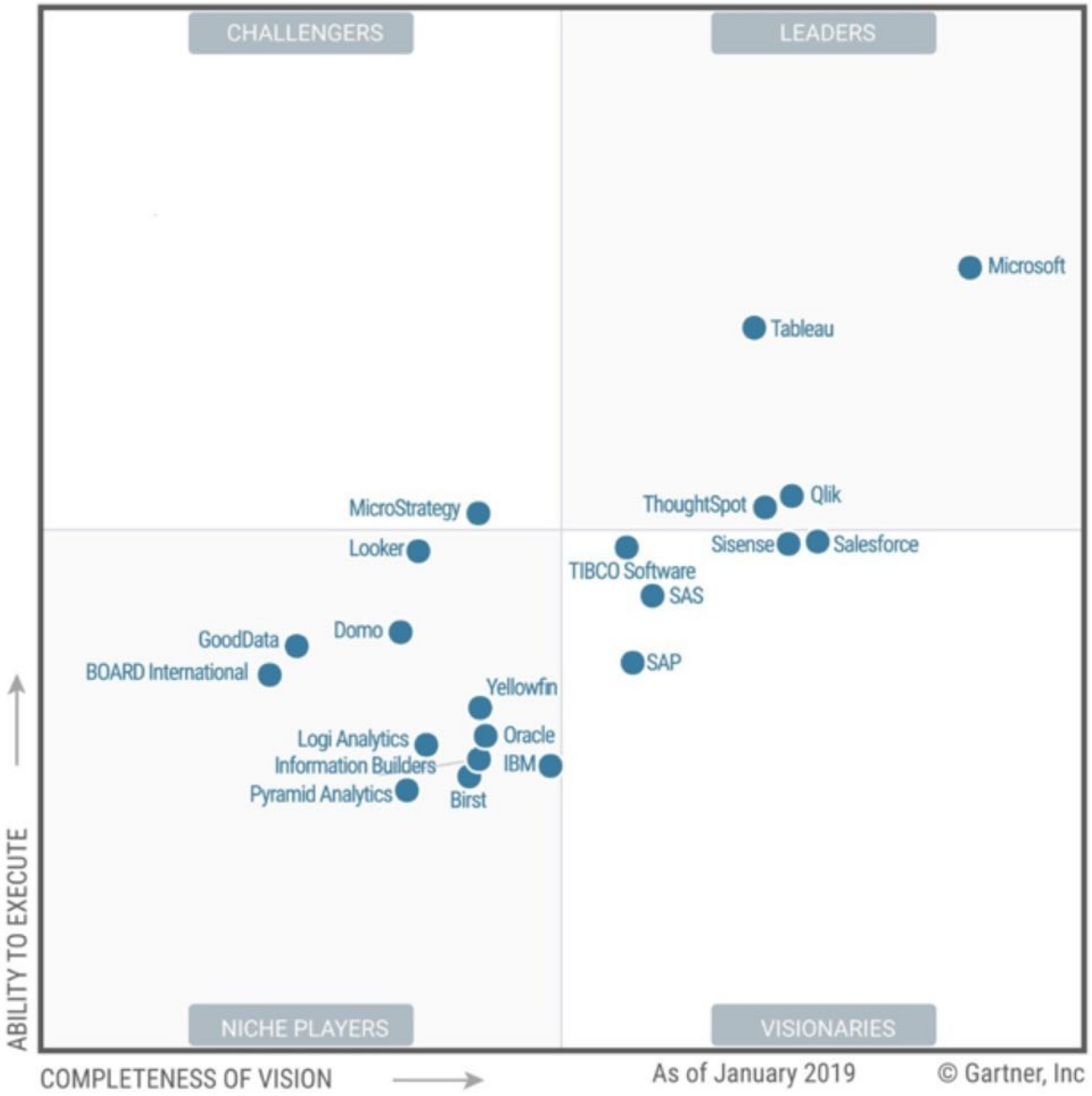
We can also consider GPU-based databases, for example:
- Kinetica on Google Cloud
  - https://www.kinetica.com/partner/google-cloud-platform/
- MapD = OmniSci
  - https://www.omnisci.com/

Similar options are available on AWS: Amazon RDS (PostgreSQL), Redshift, SnowFlake, Aurora, kinetica, MapD, etc.

Generally for sizes more than 1TB it is preferable to have a distributed DB like Google BigQuery (or Amazon SnowFlake.net)
https://statsbot.co/blog/modern-data-warehouse

Speed depends on the size of the data too.
At small sizes (<1TB) single-node solutions are faster (GPU-based are fastest).
So it maybe either one reasonably fast DB (Azure SQL Datawarehouse), or two databases - on big - but slow (Google BigQuery), and second smaller - but very fast (omnisci MapD GPU DB for interactive visual querying)
- https://fivetran.com/blog/warehouse-benchmark
- https://www.omnisci.com/demos/

Figure 1. Magic Quadrant for Analytics and Business Intelligence Platforms



Source: Gartner (February 2019)

Open Source for building dashboards

Jupyter:

- Jupyter notebooks and dashboards - https://annefou.github.io/jupyter_dashboards/02-dashboards/index.html
- Jupyter Lab - https://jupyterlab.readthedocs.io/en/stable/
- Jupyter Dashboards - https://github.com/jupyter/dashboards
- ipywidgets - https://github.com/jupyter-widgets/ipywidgets
  - https://www.youtube.com/watch?v=i40d8-Hu4vM (40 min demo from 2017)
- bqplot - https://github.com/bloomberg/bqplot
  - https://www.youtube.com/watch?v=Dmxa2Kyfzxk (demo 2018)
- Bokeh - interactive graphics - https://docs.bokeh.org/en/latest/docs/user_guide.html
- Voila - serve dashboards - https://github.com/voila-dashboards/voila
- Shiny (R-Studio) - https://shiny.rstudio.com/

Panoramix - https://github.com/teamclairvoyant/panoramix

Visualization - why so many packages?
https://www.anaconda.com/python-data-visualization-2018-why-so-many-libraries/

ipyvolume - 3-D visualization (rotate & resize 3D graphs) - https://github.com/maartenbreddels/ipyvolume

import matplotlib.pyplot as plt
import seaborn as sns

Databases optimized for analytics.

Databases can be optimized for OLTP (On-Line Transactional Processing) or OLAP (On-Line Analytical Processing). For analytics we need OLAP databases. They achieve high performance of queries by using columnar indexes, massive parallelism, taking advantage of large memory and GPU.

Columnar Indexes:
Here is a short video (4 min) made long time ago (in 2011) showing performance advantage of columnar store indexes in Microsoft SQL server.

- https://www.youtube.com/watch?v=vPN8_PCsJm4

The video demonstrates that columnar indexing is at least 40 times faster than traditional index. The query was done over a 2 Billion rows "fact" table.
Note - this video is ~8 years old. Since then MS SQL Server made lots of other performance improvements.

Note: first release of MS SQL Server was on April 24, 1989, 30 years!!

Modern hardware is much faster than that used in the above video.
Columnar store indexing is standard/default nowadays.
And of course it is available on Microsoft SQL Server on Azure cloud.
Microsoft SQL Datawarehouse on Azure cloud - fast OLAP solution.
- https://azure.microsoft.com/en-us/services/sql-data-warehouse/
Power BI - Standard Microsoft Business Intelligence tool
- https://powerbi.microsoft.com
Demo of how fast Power BI works with MS SQL Server - 500+ Mln rows of data
- https://www.youtube.com/watch?v=mQyFp2MSl-s

Demo of Kinetica DB using Tableau & Power BI:
- https://www.youtube.com/watch?v=7XliLU7ZC_s

Notice:
- sub-second response on billions of rows of data
- not need to create indexes, no wait time, fast load (even live streaming) of data

Main points:
- you can select from many different BI tools - they all support SQL
- you can select from many SQL databases - they are fast
- you can simplify your development time using standard SQL features
- you can select dedicated hardware to tune your speed
- you can achieve very fast response time
- you can enjoy lower prices

About Einstein Analytics (EA):

EA stores data not in regular SQL database, but in files and so called "inverted indexes" (II).
The motivation for selecting this technology was to have fast interactive data access.
The price to pay - the slowness of creating these "inverted indexes".
Every time you add data - you need to rebuild the whole index.
This article gives a nice explanation of what is II:
- https://www.eternussolutions.com/2016/11/13/10-things-love-wave-analytics/

And here is a picture showing the data - and its inverted index.

Row ID	Carrier	Origin	Dest	Miles	Duration
1	AA	SFO	ORD	1200	4
2	AA	SFO	PHX	350	4
3	WN	ORD	SFO	1200	4
4	WN	LAX	ATL	1500	5
5	UA	DFW	SEA	1500	5
6	UA	BOS	LAX	1500	6
7	UA	JFK	SFO	1600	6
8	UA	SFO	ORD	1200	5
9	AA	JFK	ORD	1100	5
10	DL	JFK	ORD	1100	5

Key	Value
Carrier:AA	1,2,9
Dest:ORD	1,8,9,10
Carrier:WN	3,4
Carrier:UA	5,6,7,8
Carrier:DL	10
Origin:SFO	1,2,8
etc.	

Inverted indexes are usually implemented as compressed bitmaps - and cached in memory.
They may be also implemented as "conventional" trees.
Here are more links about Inverted indexes:
- https://www.quora.com/In-database-design-what-exactly-is-the-difference-between-inverted-index-and-a-normal-index
- https://stackoverflow.com/questions/47537318/b-tree-index-vs-inverted-index
- https://en.wikipedia.org/wiki/Inverted_index

I've first worked with an SQL database with compressed bitmap indexes in 2001-2002. The database was called "Sybase IQ".
It was an SQL database for doing analytics (as opposed to transactional processing).

Since then most SQL database vendors have adopted this "columnar" indexing. They switched from row-oriented storage and indexing to compressed column-oriented storage.
This is basically equivalent to the "Inverted Index" idea.

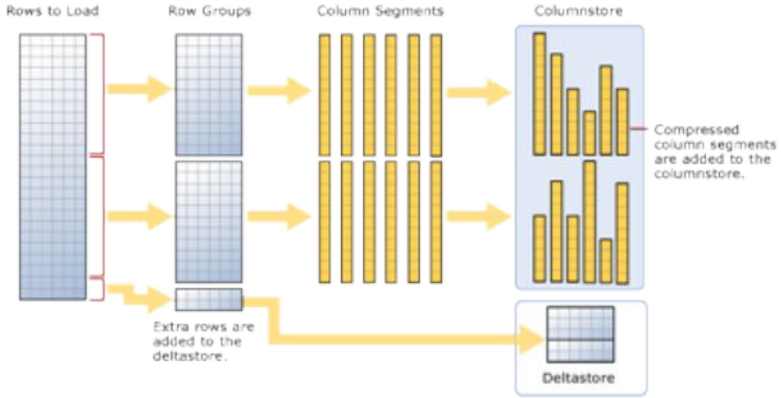
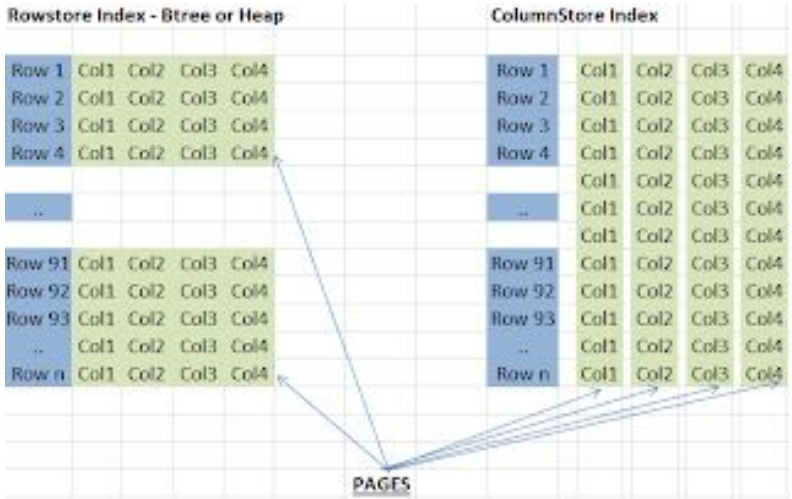
Modern SQL databases can provide performance much faster than EA. You just need to select an implementation with appropriate characteristics.

For example, on AWS (or other clouds) you can select a GPU-based SQL database on a server with terabytes of memory.
This database can query the data and give updates with a speed of a live movie (milliseconds).
And we can still use conventional ETL tools and enjoy fast data load speeds.
And it will be probably cheaper than EA.

Columnar Index = Columnstore Indexes

A column-oriented DBMS (or columnar database management system) stores data tables by column rather than by row. Mostly used for analytical queries on large tables. Query performance can be 10-40 times faster than in traditional row-accessing DBMS.

But inserts are slower (need to add and compress) - not suited for OLTP transactional databases.

Column data is of uniform type – therefore it can be compressed better than row-oriented data (using bitmap indexes and sorting).

Traditional Row-storing DB	Columnar storing DB
Excessive disk IO – read whole rows, discards most columns	Disk IO smaller because we need to read only few columns. This allows to fit more rows into memory, thus making the query much faster (all-in-memory).
Compression is not very good.	Good data compression using the fact that all data in one column has the same type.
Good random access.	Good for analytics, not so good for random access.
Can use clustered and nonclustered indexes.	Columnstore index is usually clustered. But also can use separate nonclustered indexes for some columns.
Good for transactions (OLTP)	Good for read-only analytical queries in big fact tables (OLAP)